同濟大學

TONGJI UNIVERSITY

实验报告

装

订

线

实验名称	Seq2Seq 与注意力机制
课程名称	深度学习
学　　院	计算机科学与技术学院
专　　业	数据科学与大数据技术
学生姓名	景琛皓
学　　号	2351635
指导教师	程大伟
日　　期	2026.3.25

1 Seq2Seq 实现字符串序列逆置

1.1 实验目的

- 使用 Sequence-to-Sequence（Encoder-Decoder）模型学习“将长度固定的字符串序列逆置”的映射关系。

- 掌握：Embedding → RNN Encoder → RNN Decoder → Dense 输出 logits 的标准 seq2seq 训练流程。

1.2 数据与任务定义

- 字符集： [A-Z] 共 26 个字符，并引入 0 作为 decoder 的起始 token（ <start> ）。
- 输入 enc_x ：将字符串映射为索引序列（ A→1 ... Z→26 ）。
- 标签 y ： enc_x 的逆序。
- decoder 输入 dec_x ： [0] + y[:-1] （teacher forcing）。

1.3 关键代码修改

补全 mySeq2SeqModel.call() （核心前向计算）：

```
class mySeq2SeqModel(keras.Model):
    def __init__(self):
        super(mySeq2SeqModel, self).__init__()
        self.v_sz=27
        self.embed_layer = tf.keras.layers.Embedding(self.v_sz, 64)
        self.encoder_cell = tf.keras.layers.SimpleRNNCell(128)
        self.decoder_cell = tf.keras.layers.SimpleRNNCell(128)
        self.encoder = tf.keras.layers.RNN(self.encoder_cell,
                                    return_sequences=True, return_state=True)
        self.decoder = tf.keras.layers.RNN(self.decoder_cell,
                                    return_sequences=True, return_state=True)
        self.dense = tf.keras.layers.Dense(self.v_sz)
    @tf.function
    def call(self, enc_ids, dec_ids):
        '''
        完成 sequence2sequence 模型的搭建，模块已经在`__init__`函数中定义好
        '''
        enc_emb = self.embed_layer(enc_ids)
        _, enc_state = self.encoder(enc_emb)
        dec_emb = self.embed_layer(dec_ids)
        dec_out, _ = self.decoder(dec_emb, initial_state=enc_state)
        logits = self.dense(dec_out)
        return logits
```

- enc_emb = embed(enc_ids) ：把离散 token 变成连续向量。

- _, enc_state = encoder(enc_emb) ：encoder 读完整个输入序列，得到最终隐状态 enc_state 作为“语义摘要”。

- dec_out, _ = decoder(dec_emb, initial_state=enc_state) ：decoder 以 encoder 最终状态作为初始状态，在 teacher forcing 输入 dec_emb 上生成每个时间步的隐向量序列 dec_out 。

- logits = dense(dec_out) ：把每个时间步的隐向量映射到词表大小（27）的 logits，用于 sparse_softmax_cross_entropy_with_logits 。

1.4 实验结果

训练迭代

```
optimizer = optimizers.Adam(0.0005)
model = mySeq2SeqModel()
train(model, optimizer, seqlen=20)
```

[7] ✓ 15.2s

```
step 0 : loss 3.3114936
step 500 : loss 1.6042509
step 1000 : loss 1.0736736
step 1500 : loss 0.7584256
step 2000 : loss 0.597496
step 2500 : loss 0.47932157

<tf.Tensor: shape=(), dtype=float32, numpy=0.35433117>
```

图 1.Seq2Seq 方法训练结果截图

测试模型逆置能力

首先要先对输入的一个字符串进行encode，然后在用decoder解码出逆置的字符串

测试阶段跟训练阶段的区别在于，在训练的时候decoder的输入是给定的，而在预测的时候我们需要一步步生成下一步的decoder的输入

```
def sequence_reversal():
    def decode(init_state, steps=10):
        b_sz = tf.shape(init_state[0])[0]
        cur_token = tf.zeros(shape=[b_sz], dtype=tf.int32)
        state = init_state
        collect = []
        for i in range(steps):
            cur_token, state = model.get_next_token(cur_token, state)
            collect.append(tf.expand_dims(cur_token, axis=-1))
        out = tf.concat(collect, axis=-1).numpy()
        out = [''.join([chr(idx+ord('A')-1) for idx in exp]) for exp in out]
        return out

    batched_examples, enc_x, _, _ = get_batch(32, 10)
    state = model.encode(enc_x)
    return decode(state, enc_x.get_shape()[-1]), batched_examples

def is_reverse(seq, rev_seq):
    rev_seq_rev = ''.join([i for i in reversed(list(rev_seq))])
    if seq == rev_seq_rev:
        return True
    else:
        return False
print([is_reverse(*item) for item in list(zip(*sequence_reversal()))])
print(list(zip(*sequence_reversal())))
```

[8] ✓ 0.1s Python

```
[True, True, True, True, True, True, True, True, True, True, False, True, True, True, True, True, True, True, True, True, T
[('NNTDQCQDAC', 'CADQCQDTNN'), ('CQJUOKKEAB', 'BAEKKOUJQC'), ('UXAGPMTZZH', 'HZZTMPGAXU'), ('RIJAASZOTJ', 'JTOZSAAJIR'), ('UNIMRC
```

图 2.Seq2Seq 方法测试结果截图

- 训练 loss 随步数下降，说明模型学会从输入序列预测逆序输出。
- 测试时使用 get_next_token 逐步生成输出，验证输出序列确实是输入的逆序。

2 Attentive Seq2Seq 实现字符串序列逆置

2.1 实验目的

- 在 seq2seq 基础上加入 Attention，让 decoder 在每个解码步都能“对齐/检索” encoder 的所有时间步表示，而不是只依赖 encoder 最终状态。

- 实现并理解： 双线性 attention 的打分、归一化、上下文向量计算、融合输出。

2.2 关键代码修改

（1）实现带注意力的 call()

这一段是 attentive seq2seq 的核心：计算注意力权重并形成 context，再融合进 decoder 输出。

```
class mySeq2SeqModel(keras.Model):
    def __init__(self):
        super(mySeq2SeqModel, self).__init__()
        ...
    @tf.function
    def call(self, enc_ids, dec_ids):
        '''
        todo
        完成带 attention 机制的 sequence2sequence 模型的搭建，模块已经在`__init__`函数中定义好，
        用双线性 attention，或者自己改一下`__init__`函数做加性 attention
        '''
        enc_emb = self.embed_layer(enc_ids)
        enc_out, enc_state = self.encoder(enc_emb)
        dec_emb = self.embed_layer(dec_ids)
        dec_out, _ = self.decoder(dec_emb, initial_state=enc_state)
        enc_proj = self.dense_attn(enc_out)
        score = tf.matmul(dec_out, enc_proj, transpose_b=True)
        alpha = tf.nn.softmax(score, axis=-1)
        context = tf.matmul(alpha, enc_out)
        dec_out = tf.concat([dec_out, context], axis=-1)
        logits = self.dense(dec_out)
        return logits
```

- enc_out : shape [B, T_enc, H] （encoder 每步隐藏状态）

- dec_out : shape [B, T_dec, H] （decoder 每步隐藏状态）

- enc_proj = W(enc_out) : shape [B, T_enc, H] ，相当于双线性中的矩阵 W

- score = dec_out · enc_proj^T : shape [B, T_dec, T_enc]

每个解码步对所有编码步做相似度打分

- alpha = softmax(score, axis=-1) : shape [B, T_dec, T_enc]

  对 encoder 维度归一化，得到注意力权重

- context = alpha · enc_out : shape [B, T_dec, H]

  得到每个解码步对应的上下文向量（对 encoder 信息的加权和）

- concat([dec_out, context]) : shape [B, T_dec, 2H]

  把“解码器当前信息”与“检索到的编码器信息”融合，再输出到词表 logits。

（2）实现推理阶段单步解码 get_next_token()

训练时 decoder 输入是给定的（teacher forcing），推理时必须一步步生成。因此需要单步 attention 解码逻辑。

```
@tf.function
    def encode(self, enc_ids):
        enc_emb = self.embed_layer(enc_ids) # shape(b_sz, len, emb_sz)
        enc_out, enc_state = self.encoder(enc_emb)
        return enc_out, [enc_out[:, -1, :], enc_state]
    def get_next_token(self, x, state, enc_out):
        '''
        todo
        参考 sequence_reversal-exercise, 自己构建单步解码逻辑
        '''
        inp_emb = self.embed_layer(x)
        if isinstance(state, (list, tuple)):
            dec_state = state[1] if len(state) == 2 else state[0]
        else:
            dec_state = state
        h, new_state = self.decoder_cell.call(inp_emb, [dec_state])
        enc_proj = self.dense_attn(enc_out)
        score = tf.matmul(tf.expand_dims(h, axis=1), enc_proj, transpose_b=True)
        alpha = tf.nn.softmax(score, axis=-1)
        context = tf.matmul(alpha, enc_out)
        context = tf.squeeze(context, axis=1)
        logits = self.dense(tf.concat([h, context], axis=-1))
        out = tf.argmax(logits, axis=-1, output_type=tf.int32)
        state = [h, new_state[0]]
        return out, state
```

- decoder_cell.call(...) 做单步 RNN 更新，得到当前隐状态 h 。
- 使用 h 作为 query，对 enc_out 做一次 attention，得到单步 context 。
- dense(concat([h, context])) 得到词表 logits， argmax 得到预测 token。
- 更新并返回新的 state，供下一步解码继续使用。

2.3 实验结果

训练迭代

```
optimizer = optimizers.Adam(0.0005)
model = mySeq2SeqModel()
train(model, optimizer, seqlen=20)
```

[15] ✓ 12.6s

```
step 0 : loss 3.296161
step 500 : loss 1.718017
step 1000 : loss 0.17472605
step 1500 : loss 0.06483354

<tf.Tensor: shape=(), dtype=float32, numpy=0.0133610265>
```

图 3.Attentive Seq2Seq 方法训练结果截图

测试模型逆置能力

首先要先对输入的一个字符串进行encode，然后在用decoder解码出逆置的字符串

测试阶段跟训练阶段的区别在于，在训练的时候decoder的输入是给定的，而在预测的时候我们需要一步步生成下一步的decoder的输入

```python
def sequence_reversal():
    def decode(init_state, steps, enc_out):
        b_sz = tf.shape(init_state[0])[0]
        cur_token = tf.zeros(shape=[b_sz], dtype=tf.int32)
        state = init_state
        collect = []
        for i in range(steps):
            cur_token, state = model.get_next_token(cur_token, state, enc_out)
            collect.append(tf.expand_dims(cur_token, axis=-1))
        out = tf.concat(collect, axis=-1).numpy()
        out = [''.join([chr(idx+ord('A')-1) for idx in exp]) for exp in out]
        return out

    batched_examples, enc_x, _, _ = get_batch(32, 20)
    enc_out, state = model.encode(enc_x)
    return decode(state, enc_x.get_shape()[-1], enc_out), batched_examples

def is_reverse(seq, rev_seq):
    rev_seq_rev = ''.join([i for i in reversed(list(rev_seq))])
    if seq == rev_seq_rev:
        return True
    else:
        return False
print([is_reverse(*item) for item in list(zip(*sequence_reversal()))])
print(list(zip(*sequence_reversal())))
```

6] ✓ 0.2s Python

```
[True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, Tr
[('AMCHJVJRWIJAYERNEHUQ', 'QUHENREYAJIWRJVJHCMA'), ('DGPXKCEPBBTHEYKUGACP', 'PCAGUKYEHTBBPECKXPGD'), ('TLIJRGOKDSNXNFAFPLJV', 'JV
```

图 4.Attentive Seq2Seq 方法测试结果截图

- attentive 模型理论上更容易学习“对齐关系”：第 t 个解码步应更多关注 encoder 的第 T-1-t 个位置（逆序对齐）。

- 在更复杂任务（可变长、噪声、长序列）上，attention 往往比纯 encoder 最终状态更稳定、收敛更快。